

Unser eigener KI-Lernagent

Schritt für Schritt bis zum modernen Webinterface

Bearbeitbare Kursfassung · Version 1.1 · August 2026 · didaktisch
erweitert

Kursübersicht

So arbeitest du mit jedem Kapitel

1 · Ziel verstehen: Lies zuerst, was du am Ende erklären oder zeigen kannst. **2 · Jetzt tun:** Arbeite den konkreten Auftrag im Repository ab. **3 · KI als Denkcoach:** Nutze den Startprompt, beantworte Rückfragen und prüfe jeden Vorschlag selbst.

Kapitel-Navigation

Tippe in der PDF auf ein Kapitel, um direkt dorthin zu springen. *Im Word-Dokument funktioniert der Link mit Strg + Klick.*

[1. Das Problem vor der Technik](#)

[2. Was ist ein Agent?](#)

[3. Zielarchitektur und Datenfluss](#)

[4. GitHub als Lernwerkstatt](#)

[5. UI-Prototyp: zuerst sichtbar denken](#)

[6. Frontend mit React und TypeScript](#)

[7. Backend und erste API](#)

[8. Der didaktische Tutor](#)

[9. LLM-Adapter statt Anbieterbindung](#)

[10. Lokale Wissensbasis und RAG](#)

[11. Relationale Datenbank](#)

[12. Wissensgraph und Graphentheorie](#)

[13. Programmierwissen modular organisieren](#)

[14. Sichere Code-Ausführung](#)

[15. Sicherheit gegen Prompt Injection](#)

[16. Feedback, Lernen und Evaluation](#)

[17. Webinterface bis zur Produktreife](#)

[18. Abschlussprojekt und Präsentation](#)

Tipp: Nutze zusätzlich die PDF-Lesezeichen für die Kapitelübersicht.

Die PDF-Fassung enthält die optimierte mobile A5-Navigation. Diese Word-Fassung dient der Weiterbearbeitung und Anpassung an den Unterricht.

1. Das Problem vor der Technik

DEIN KAPITELSTART

Ziel: Ein konkretes Lernproblem auswählen und als überprüfbare User Story beschreiben.

Jetzt tun: Öffne das Repository, lege das Issue „Lernproblem“ an und ergänze drei typische Lernsituationen sowie passende Akzeptanzkriterien.

Möglicher Startprompt: „Ich entwickle einen Lernagenten für Programmieranfänger. Stelle mir nacheinander fünf Fragen, damit ich aus einem konkreten Lernproblem eine User Story und überprüfbare Akzeptanzkriterien formuliere. Gib noch keine fertige Lösung vor.“

Warum bauen wir den Agenten?

Lernende verlieren sich häufig zwischen Fehlermeldung, Syntax und fertigen KI-Lösungen. Unser Agent soll den nächsten sinnvollen Denkschritt sichtbar machen, Vorwissen aktivieren und niemals ungefragt die ganze Lösung vorwegnehmen.

Lernziele

Du kannst Nutzerbedarf, Lernziel und technische Funktion unterscheiden. Du formulierst eine User Story und prüfbare Akzeptanzkriterien.

Auftrag

Lege in GitHub ein Issue „Lernproblem“ an. Beschreibe drei typische Situationen: Verständnisfrage, Fehlersuche und Übungswunsch. Ergänze jeweils: Was soll die lernende Person danach selbst können?

GitHub-Checkpoint: Ordne die Arbeit einem Issue zu, committe einen nachvollziehbaren Zwischenstand, pushe ihn und öffne einen Pull Request mit Test- oder Prüfnachweis. → Begriffe im GitHub-Glossar

2. Was ist ein Agent?

DEIN KAPITELSTART

Ziel: Den Unterschied zwischen Chatbot, LLM und Agent erklären und den Agentenloop skizzieren.

Jetzt tun: Zeichne den Agentenloop mit fünf Schritten und ordne jedem Schritt ein Beispiel aus eurem Lernagenten zu.

Möglicher Startprompt: „Erkläre mir Chatbot, LLM und KI-Agent mit demselben Alltagsbeispiel. Frage mich anschließend, welcher Baustein in unserem Lernagenten welche Aufgabe übernimmt.“

Denkmodell

Ein Chatbot erzeugt Text. Ein Agent erhält ein Ziel, verwaltet Zustand, wählt erlaubte Werkzeuge, beobachtet Ergebnisse und entscheidet über den nächsten Schritt. Der Agentenloop lautet: verstehen → planen → Werkzeug wählen → Ergebnis prüfen → antworten.

Bausteine

UI, API, Orchestrator, LLM-Adapter, Werkzeuge, Gedächtnis, Wissensbasis, Regeln/Guardrails, Protokollierung und Evaluation.

Training richtig verstehen

Im Basiskurs „trainieren“ Schüler den Agenten durch geprüfte Quellen, Beispiele, Feedback, Prompts und Tests. Modell-Fine-Tuning ist eine spätere, getrennte Vertiefung. Falsche Rückmeldungen dürfen nicht automatisch zu Wissen werden.

GitHub-Checkpoint: Ordne die Arbeit einem Issue zu, committe einen nachvollziehbaren Zwischenstand, pushe ihn und öffne einen Pull Request mit Test- oder Prüfnachweis. → Begriffe im GitHub-Glossar

3. Zielarchitektur und Datenfluss

DEIN KAPITELSTART

Ziel: Den Weg einer Schülerfrage durch Frontend, Backend, Wissenssuche, LLM und Datenhaltung nachvollziehen.

Jetzt tun: Übertrage die Zielarchitektur in eine eigene Skizze und

beschrifte jeden Datenübergang mit Eingabe und Ausgabe.

Möglicher Startprompt: „Hilf mir, den Datenfluss unseres Lernagenten Schritt für Schritt zu prüfen. Beginne bei einer Frage im Webinterface und stelle mir pro Station eine Verständnisfrage.“

End-to-End

Die React-Oberfläche sendet eine Nachricht an FastAPI. Der Orchestrator klassifiziert die Lernabsicht, sucht lokale Quellen, ruft bei Bedarf ein Werkzeug auf und bittet den LLM-Adapter um eine strukturierte Antwort. SQLite speichert Sitzungen, Quellen, Aufgaben und Feedback; der Wissensgraph speichert Beziehungen.

Ordnerstruktur

frontend/ für React und TypeScript; backend/app/ für API, Domain und Adapter; knowledge/ für geprüfte Lernmaterialien; tests/ für automatische Prüfungen; docs/ für Entscheidungen.

Architekturregel

Kein API-Schlüssel im Browser. Das Frontend spricht nur mit dem eigenen Backend. Jeder austauschbare Anbieter implementiert dieselbe LLM-Schnittstelle.

GitHub-Checkpoint: Ordne die Arbeit einem Issue zu, committe einen nachvollziehbaren Zwischenstand, pushe ihn und öffne einen Pull Request mit Test- oder Prüfnachweis. → Begriffe im GitHub-Glossar

4. GitHub als Lernwerkstatt

DEIN KAPITELSTART

Ziel: GitHub nicht nur als Ablage, sondern als nachvollziehbaren Arbeits- und Rückmeldeprozess nutzen.

Jetzt tun: Wähle ein Issue, erstelle einen passenden Branch, ändere genau eine kleine Sache und dokumentiere sie mit Commit und Pull Request.

Möglicher Startprompt: „Begleite mich als GitHub-Coach von einem Issue bis zum Pull Request. Frage nach jedem Schritt, was ich getan

habe und warum. Erkläre unbekannte Begriffe kurz, bevor ich sie anwende.“

Workflow

Repository aus Vorlage erzeugen → Issue wählen (Aufgabe und Ziel klären) → Branch erstellen (Änderung vom Hauptstand trennen) → kleine Änderung committen (Zwischenstand speichern) → pushen (Commits auf GitHub übertragen) → Tests ausführen → Pull Request öffnen (Änderung zur gemeinsamen Prüfung vorschlagen) → Feedback einarbeiten.

Merksatz: Das Issue erklärt das Warum, der Branch trennt die Arbeit, der Commit dokumentiert den Schritt, der Push macht ihn sichtbar und der Pull Request eröffnet die gemeinsame Prüfung.

Commit-Beispiele

feat(ui): add message composer; fix(api): validate empty prompt;
test(tutor): add hint-level case; docs(graph): explain prerequisite edge.

Definition of Done

Die Funktion ist sichtbar bedienbar, Backend und Frontend arbeiten zusammen, Tests laufen, keine Geheimnisse sind committed, README und Screenshot sind aktualisiert.

GitHub-Checkpoint: Ordne die Arbeit einem Issue zu, committe einen nachvollziehbaren Zwischenstand, pushe ihn und öffne einen Pull Request mit Test- oder Prüfnachweis. → Begriffe im GitHub-Glossar

5. UI-Prototyp: zuerst sichtbar denken

DEIN KAPITELSTART

Ziel: Ein übersichtliches, responsives Agenteninterface als sichtbaren Prototyp planen.

Jetzt tun: Skizziere Desktop- und Mobilansicht, markiere Chat, Navigation, Quellen, Lernziel und Eingabefeld und begründe die Reihenfolge.

Möglicher Startprompt: „Stelle mir sechs UX-Fragen für ein

schülergerechtes Agenten-Webinterface. Hilf mir danach, aus meinen Antworten eine Komponentenliste und Akzeptanzkriterien abzuleiten.“

Standardlayout

Desktop: linke Seitenleiste für Kurs/Verlauf, mittlere Chatfläche, rechte Kontextleiste für Lernziel und Quellen. Mobil: eine Spalte; Seitenleisten öffnen als Dialog/Drawer. Unten bleibt der Composer gut erreichbar.

Kernkomponenten

AppShell, CourseNav, Conversation, Message, CodeBlock, SourceCard, HintControls, Composer, LearningProgress und Settings.

UX-Regeln

Semantisches HTML, Tastaturbedienung, sichtbarer Fokus, ausreichender Kontrast, Statusmeldungen via aria-live, responsive Layout, verständliche Lade- und Fehlerzustände. Keine Farbe als einziges Signal.

GitHub-Checkpoint: Ordne die Arbeit einem Issue zu, committe einen nachvollziehbaren Zwischenstand, pushe ihn und öffne einen Pull Request mit Test- oder Prüfnachweis. → Begriffe im GitHub-Glossar

6. Frontend mit React und TypeScript

DEIN KAPITELSTART

Ziel: Den UI-Prototyp in typisierte, wiederverwendbare React-Komponenten überführen.

Jetzt tun: Erzeuge das Projekt, implementiere zuerst Message und Composer mit Beispieldaten und prüfe die Mobilansicht.

Möglicher Startprompt: „Hilf mir, die Komponente Message in React und TypeScript selbst zu entwickeln. Beginne mit den benötigten Daten und gib mir nur den nächsten kleinen Schritt, nicht sofort den vollständigen Code.“

Start

Erzeuge ein React/TypeScript-Projekt mit einem aktuellen Build-Werkzeug. Komponenten erhalten typisierte Props; API-Zustand und UI-Zustand bleiben getrennt.

Datenmodell

Eine Nachricht besitzt id, role, content, createdAt, citations, codeBlocks und tutorStage. Die UI rendert Daten – sie entscheidet nicht über Tutorlogik.

Checkpoint

Statische Beispielnachrichten darstellen; Dark Mode nicht priorisieren, bevor Tastatur, Mobilansicht und Fehlerzustände funktionieren.

GitHub-Checkpoint: Ordne die Arbeit einem Issue zu, committe einen nachvollziehbaren Zwischenstand, pushe ihn und öffne einen Pull Request mit Test- oder Prüfnachweis. → Begriffe im GitHub-Glossar

7. Backend und erste API

DEIN KAPITELSTART

Ziel: Eine validierte Schnittstelle zwischen Webinterface und Agentenlogik bereitstellen.

Jetzt tun: Implementiere zuerst /api/health, danach eine Mock-Antwort für POST /api/chat und teste gültige sowie ungültige Eingaben.

Möglicher Startprompt: „Führe mich in kleinen Schritten zu einer FastAPI-Route POST /api/chat. Frage zuerst nach Request und Response und lass mich die Schemas selbst entwerfen.“

FastAPI

Die Route POST /api/chat nimmt eine validierte Nachricht entgegen und liefert zunächst eine simulierte Tutorantwort. GET /api/health zeigt den Betriebszustand.

Vertrag

Request und Response werden mit Schemas typisiert. Fehler besitzen Statuscode, verständliche Meldung und request_id. CORS ist nur für bekannte Entwicklungs-/Produktionsursprünge erlaubt.

Test

Teste leere Nachricht, zu lange Nachricht, gültige Frage und Serverfehler. Verbinde danach den Composer mit fetch und AbortController.

GitHub-Checkpoint: Ordne die Arbeit einem Issue zu, committe einen nachvollziehbaren Zwischenstand, pushe ihn und öffne einen Pull Request mit Test- oder Prüfnachweis. → Begriffe im GitHub-Glossar

8. Der didaktische Tutor

DEIN KAPITELSTART

Ziel: Hilfen so staffeln, dass Lernende weiterdenken, statt sofort eine Komplettlösung zu erhalten.

Jetzt tun: Formuliere zu einem typischen Programmierfehler alle fünf Stufen der Tutorleiter und teste sie mit einer Mitschülerin oder einem Mitschüler.

Möglicher Startprompt: „Ich habe einen Programmierfehler. Nutze die Tutorleiter: Beginne mit einer Diagnosefrage und erhöhe die Hilfestufe nur auf meine Bitte. Fordere mich am Ende zu einer Warum-Erklärung auf.“

Tutorleiter

1 Diagnosefrage, 2 Denkhinweis, 3 Teilbeispiel, 4 Lösungsskizze, 5 vollständige Lösung mit Reflexion. Lernende können „kleiner Hinweis“, „mehr erklären“ oder „Lösung prüfen“ wählen.

Antwortstruktur

Kurzdiagnose → nächste Frage/Aktion → optionaler Hinweis → Code nur wenn nötig → Quelle → Mini-Check „Warum funktioniert das?“

MindLink

Neue Begriffe werden an Bekanntes angebunden: PHP-Array ↔ JavaScript-Array; Java-Klasse ↔ TypeScript-Klasse; SQL-Beziehung ↔ Graphkante.

GitHub-Checkpoint: Ordne die Arbeit einem Issue zu, committe einen nachvollziehbaren Zwischenstand, pushe ihn und öffne einen Pull Request mit Test- oder Prüfnachweis. → Begriffe im GitHub-Glossar

9. LLM-Adapter statt Anbieterbindung

DEIN KAPITELSTART

Ziel: Eine neutrale Schnittstelle entwerfen, damit das Sprachmodell austauschbar bleibt.

Jetzt tun: Definiere Eingabe, Ausgabe und Fehlerfälle des Adapters und implementiere zunächst einen MockAdapter ohne echten API-Aufruf.

Möglicher Startprompt: „Hilf mir, eine anbieterneutrale LLM-Schnittstelle zu entwerfen. Stelle zuerst Fragen zu Eingabe, Ausgabe, Zeitüberschreitung und Validierung; zeige Code erst nach meinem Entwurf.“

Schnittstelle

`generate(messages, tools, response_schema)` liefert ein neutrales Ergebnis. Implementierungen können Cloud- oder lokale Modelle ansprechen. Konfiguration kommt ausschließlich vom Server.

Strukturierte Ausgabe

Der Agent fordert JSON mit `answer`, `next_question`, `hint_level`, `citations` und `suggested_action`. Das Backend validiert das Ergebnis, bevor es an die UI geht.

Fallback

Bei Zeitüberschreitung oder ungültiger Ausgabe zeigt die UI eine ehrliche Fehlermeldung und bietet Wiederholen an. Das System erfindet keine Quelle.

GitHub-Checkpoint: Ordne die Arbeit einem Issue zu, committe einen nachvollziehbaren Zwischenstand, pushe ihn und öffne einen Pull Request mit Test- oder Prüfnachweis. → Begriffe im GitHub-Glossar

10. Lokale Wissensbasis und RAG

DEIN KAPITELSTART

Ziel: Geprüfte Kursquellen auffindbar machen und Antworten mit nachvollziehbaren Belegen erzeugen.

Jetzt tun: Wähle ein eigenes Lernmaterial, erfasse Herkunft und Lizenz, zerlege es in sinnvolle Abschnitte und teste drei Suchfragen.

Möglicher Startprompt: „Hilf mir, ein Kursdokument für RAG vorzubereiten. Frage zuerst nach Lernziel, Quelle, Lizenz und sinnvoller Abschnittsgröße und prüfe danach meine Metadaten.“

Pipeline

Dokumente auswählen → Lizenz/Quelle notieren → in kleine Abschnitte teilen → Metadaten speichern → Embeddings erzeugen → passende Abschnitte abrufen → Antwort mit Quellen erzeugen.

Quellenqualität

Bevorzugt werden offizielle Dokumentationen und selbst erstellte Kursmaterialien. Jeder Chunk speichert Titel, URL/Datei, Sprache, Version, Prüfdatum und Themen.

Schülerbeitrag

Neue Inhalte gelangen als Vorschlag in eine Review-Warteschlange. Erst nach fachlicher Prüfung werden sie freigegeben.

GitHub-Checkpoint: Ordne die Arbeit einem Issue zu, committe einen nachvollziehbaren Zwischenstand, pushe ihn und öffne einen Pull Request mit Test- oder Prüfnachweis. → Begriffe im GitHub-Glossar

11. Relationale Datenbank

DEIN KAPITELSTART

Ziel: Strukturierte Daten des Lernagenten in Tabellen, Schlüssel und Beziehungen übersetzen.

Jetzt tun: Modelliere Benutzer, Sitzung, Nachricht und Quelle; begründe Primär- und Fremdschlüssel und teste eine 1:n-Beziehung.

Möglicher Startprompt: „Sei mein Datenbank-Coach. Gib mir eine kleine Anforderung für unseren Lernagenten und lass mich daraus Tabellen, Primärschlüssel und Fremdschlüssel ableiten. Korrigiere mit Hinweisen.“

Warum SQL?

Tabellen eignen sich für klar strukturierte, konsistente Daten: Benutzer, Kurse, Sitzungen, Nachrichten, Aufgaben, Quellen, Feedback und Freigaben.

Beziehungen

user 1:n session; session 1:n message; message n:m source über message_source; exercise 1:n attempt. Fremdschlüssel sichern gültige Verweise.

Datenschutz

So wenig personenbezogene Daten wie möglich; Pseudonyme, Rollen, Löschfristen und getrennte Inhalts-/Telemetriedaten. Keine sensiblen Schülerdaten in Modellprompts.

GitHub-Checkpoint: Ordne die Arbeit einem Issue zu, committe einen nachvollziehbaren Zwischenstand, pushe ihn und öffne einen Pull Request mit Test- oder Prüfnachweis. → Begriffe im GitHub-Glossar

12. Wissensgraph und Graphentheorie

DEIN KAPITELSTART

Ziel: Lerninhalte als Knoten und Beziehungen modellieren und daraus erklärbare Lernwege ableiten.

Jetzt tun: Erstelle einen Mini-Graphen aus fünf Programmierbegriffen,

kennzeichne Voraussetzungen und finde mindestens einen Lernpfad.

Möglicher Startprompt: „Hilf mir, aus fünf Begriffen einen Wissensgraphen zu bauen. Frage mich zuerst nach Knoten und Kantentypen und prüfe anschließend, ob Voraussetzungen, Pfade oder Zyklen sinnvoll sind.“

Warum ein Graph?

Lernwissen ist vernetzt. Knoten sind Konzepte, Aufgaben, Sprachen und Quellen. Kanten heißen zum Beispiel `requires`, `example_of`, `contrasts_with` oder `taught_by`.

Graphbegriffe

Pfad: Lernweg zwischen Konzepten. Grad: Zahl der Verbindungen.

Zyklus: möglicherweise problematische Voraussetzungsschleife.

Gewicht: Stärke oder Sicherheit einer Beziehung.

LLM-Verbindung

Der Graph bestimmt nicht die Formulierung, sondern liefert erklärbare

Struktur: Welche Voraussetzungen fehlen? Welches Beispiel verbindet Bekanntes mit Neuem? Die gefundenen Knoten werden als Kontext an das LLM gegeben.

GitHub-Checkpoint: Ordne die Arbeit einem Issue zu, committe einen nachvollziehbaren Zwischenstand, pushe ihn und öffne einen Pull Request mit Test- oder Prüfnachweis. → Begriffe im GitHub-Glossar

13. Programmierwissen modular organisieren

DEIN KAPITELSTART

Ziel: Sprachübergreifende Konzepte mit Voraussetzungen, Beispielen, Übungen und Quellen als Lernmodule strukturieren.

Jetzt tun: Wähle ein Konzept, etwa Schleife oder Klasse, und erstelle dafür eine vollständige Konzeptkarte für zwei Programmiersprachen.

Möglicher Startprompt: „Hilf mir, eine Konzeptkarte zum Thema

[Thema einsetzen] zu erstellen. Frage nach Vorwissen, typischen Fehlvorstellungen, Minimalbeispiel, Gegenbeispiel, Übung und Test.“

Fundament

HTML: Semantik und Formulare; CSS: Boxmodell, Layout, Responsive Design; JavaScript/TypeScript: Daten, Funktionen, DOM, async; PHP: Request, Session, Datenbank; Java: OOP, Typen, Tests.

Frameworks

Framework-Wissen ist versioniert und austauschbar. Zuerst werden Sprach- und Webgrundlagen gelernt, danach etwa React sowie serverseitige Frameworks. Der Agent kennzeichnet versionsabhängige Aussagen.

Konzeptmatrix

Jedes Thema erhält Voraussetzungen, Fehlvorstellungen, Minimalbeispiel, Gegenbeispiel, Übung, Test und geprüfte Quellen.

GitHub-Checkpoint: Ordne die Arbeit einem Issue zu, committe einen nachvollziehbaren Zwischenstand, pushe ihn und öffne einen Pull Request mit Test- oder Prüfnachweis. → Begriffe im GitHub-Glossar

14. Sichere Code-Ausführung

DEIN KAPITELSTART

Ziel: Verstehen, warum fremder Code isoliert ausgeführt werden muss, und Schutzgrenzen entwerfen.

Jetzt tun: Erstelle ein Bedrohungsmodell und eine Checkliste für Netzwerk, Dateien, Laufzeit, Speicher, Prozesse und Ausgabe.

Möglicher Startprompt: „Prüfe mit mir ein Sandbox-Konzept für Schülercode. Stelle zu jeder Ressource eine Risiko-Frage und fordere eine konkrete technische Begrenzung.“

Grundsatz

Niemals untrusted Schülercode direkt im Agentenprozess oder auf dem Schulserver ausführen. HTML/CSS-Vorschau wird strikt isoliert;

JavaScript, PHP, Python oder Java benötigen getrennte Sandbox-Container.

Grenzen

Kein Netzwerk, schreibgeschütztes Basissystem, temporäres Arbeitsverzeichnis, CPU-/RAM-/Zeitlimit, Prozesslimit, erlaubte Images, Ausgabegrößenlimit und vollständige Löschung nach dem Lauf.

Freigabe

Der Agent darf Code vorschlagen. Eine Ausführung ist eine explizite Benutzeraktion. Gefährliche Werkzeugaufrufe benötigen Regelprüfung und gegebenenfalls Lehrkraftfreigabe.

GitHub-Checkpoint: Ordne die Arbeit einem Issue zu, committe einen nachvollziehbaren Zwischenstand, pushe ihn und öffne einen Pull Request mit Test- oder Prüfnachweis. → Begriffe im GitHub-Glossar

15. Sicherheit gegen Prompt Injection

DEIN KAPITELSTART

Ziel: Anweisungen des Systems von unvertrauenswürdigen Inhalten unterscheiden und Angriffe testen.

Jetzt tun: Lege mindestens fünf Angriffstests an und dokumentiere erwartetes sicheres Verhalten sowie tatsächliches Ergebnis.

Möglicher Startprompt: „Erstelle mit mir schrittweise Prompt-Injection-Testfälle für einen Lernagenten. Verrate keine Geheimnisse und erkläre nach jedem Test, welche Schutzschicht geprüft wird.“

Bedrohung

Webseiten, Dateien oder Schülercode können Text enthalten, der den Agenten zu Regelbruch oder Datenabfluss bewegen soll. Externe Inhalte sind Daten, keine vertrauenswürdigen Anweisungen.

Schutzschichten

Systemregeln, klare Datengrenzen, Tool-Allowlist, minimale Rechte, Eingabe-/Ausgabvalidierung, Geheimnisverwaltung, menschliche Freigaben und Angriffs-Tests.

Testfälle

„Ignoriere alle Regeln“, versteckte Anweisung in einer Quelle, SQL-Injection, XSS im Chat, riesige Eingabe, manipulierte Quellenangabe und Versuch, API-Schlüssel auszugeben.

GitHub-Checkpoint: Ordne die Arbeit einem Issue zu, committe einen nachvollziehbaren Zwischenstand, pushe ihn und öffne einen Pull Request mit Test- oder Prüfnachweis. → Begriffe im GitHub-Glossar

16. Feedback, Lernen und Evaluation

DEIN KAPITELSTART

Ziel: Die Lernwirkung mit festen Testfällen und sinnvollen Kennzahlen statt mit Bauchgefühl bewerten.

Jetzt tun: Erstelle ein kleines Evaluationsset mit je einer Fach-, Didaktik-, Quellen- und Sicherheitsprüfung und definiere Bestehensregeln.

Möglicher Startprompt: „Hilf mir, messbare Qualitätskriterien für unseren Lernagenten zu formulieren. Hinterfrage jede Kennzahl: Was sagt sie wirklich über Lernen aus und was nicht?“

Feedbacktypen

Hilfreich/nicht hilfreich, Fehlergrund, gewünschte Hinweistiefe, Quellenproblem und Verbesserungsvorschlag. Feedback wird nicht ungeprüft zur neuen Wahrheit.

Evaluationsset

Baue feste Testfragen zu HTML, CSS, JavaScript, PHP und Java: korrekte Antwort, sinnvolle Rückfrage, Quellenpassung, keine Komplettlösung zu früh, sichere Toolwahl.

Kennzahlen

Aufgabenabschluss, benötigte Hinweise, selbst erklärte Lösung, Quellengenauigkeit, Fehlerquote und Sicherheitstests – nicht bloß Zahl der Chatnachrichten.

GitHub-Checkpoint: Ordne die Arbeit einem Issue zu, committe einen nachvollziehbaren Zwischenstand, pushe ihn und öffne einen Pull Request mit Test- oder Prüfnachweis. → Begriffe im GitHub-Glossar

17. Webinterface bis zur Produktreife

DEIN KAPITELSTART

Ziel: Einzelne Funktionen zu einem zugänglichen, robusten und verständlichen Lernprodukt verbinden.

Jetzt tun: Wähle die nächste Ausbaustufe, ergänze Lade-, Leer- und Fehlerzustand und teste Tastatur sowie Smartphone-Ansicht.

Möglicher Startprompt: „Führe mit mir einen Produktreife-Check für unser Agenteninterface durch. Frage nacheinander nach Bedienbarkeit, Mobilansicht, Barrierefreiheit, Fehlerzuständen und verständlichen Statusmeldungen.“

Ausbaustufen

A statische UI; B Mock-API; C echter Chat; D Streaming; E Quellen/Code; F Lernmodus; G Verlauf/SQL; H Graphansicht; I Sandbox; J Lehrkraft-Dashboard.

Agenten-typische Zustände

Bereit, denkt, sucht Quelle, prüft Code, wartet auf Freigabe, abgeschlossen, Fehler. Zeige nur nachvollziehbare Statusmeldungen – keine erfundenen Gedankengänge.

Mobile-first

Composer oberhalb der Bildschirmtastatur, große Touch-Ziele, kurze Quellenkarten, Code horizontal scrollbar, Navigation als Drawer, kein dreispaltiges Desktoplayout auf dem Handy.

GitHub-Checkpoint: Ordne die Arbeit einem Issue zu, committe einen nachvollziehbaren Zwischenstand, pushe ihn und öffne einen Pull Request mit Test- oder Prüfnachweis. → Begriffe im GitHub-Glossar

18. Abschlussprojekt und Präsentation

DEIN KAPITELSTART

Ziel: Den Lernagenten anhand eines vollständigen Nutzungsszenarios prüfen, präsentieren und ehrlich einordnen.

Jetzt tun: Führe Fach-, UX- und Sicherheitstests aus, dokumentiere Grenzen und bereite eine kurze Demo vom Problem bis zur Reflexion vor.

Möglicher Startprompt: „Sei mein kritischer Abnahme-Coach. Prüfe unser Abschlussprojekt anhand von Funktion, Lernwirkung, Quellen, Datenschutz, Sicherheit und Barrierefreiheit. Stelle immer nur eine Frage.“

Pflichtumfang

Eine lernende Person stellt eine Programmierfrage, erhält eine stufenweise Hilfe mit Quelle, sendet Code zur Prüfung, sieht Feedback und findet den Verlauf wieder.

Abnahme

Fünf Fachtests, fünf UX-Tests, fünf Sicherheitstests;
Lighthouse/Accessibility-Check; Pull Request mit
Architekturentscheidung und kurzem Demo-Video.

Reflexion

Was kann der Agent verlässlich? Wo braucht er Lehrkraftkontrolle?
Welche Daten benötigt er wirklich? Welche Verbesserung hätte den
größten Lernnutzen?

GitHub-Checkpoint: Ordne die Arbeit einem Issue zu, committe einen nachvollziehbaren Zwischenstand, pushe ihn und öffne einen Pull Request mit Test- oder Prüfnachweis. → Begriffe im GitHub-Glossar

12-Sprint-Kursplan

Sprint	Produktinkrement	Nachweis
1	Problem, User Stories, Repo	Issues + README

2	Responsive UI-Prototyp	Screenshot + UX-Test
3	React-Komponenten	Komponententests
4	FastAPI + Mock-Chat	API-Tests
5	Tutorleiter + Schemas	didaktische Tests
6	LLM-Adapter	Provider-Wechseltest
7	RAG + Quellen	Quellenprüfung
8	SQLite + Verlauf	Migration + SQL-Test
9	Wissensgraph	Lernpfad-Abfrage
10	Sandbox-Konzept	Threat Model
11	Evaluation + A11y	Testbericht
12	Demo + Reflexion	Release v1.0

Begriffsindex: GitHub, Agent und KI

So nutzt du den Index: Schlage einen Begriff nach, formuliere ihn anschließend mit eigenen Worten und nenne ein Beispiel aus eurem Repository. Die Spalte „Wozu?“ erklärt, welches Problem der Begriff löst.

Begriff	Bedeutung	Wozu gibt es das?
Acceptance Criteria / Akzeptanzkriterien	Prüfbare Bedingungen, an denen erkennbar ist, ob eine Aufgabe erfüllt wurde.	Sie machen aus einer Idee ein kontrollierbares Ergebnis.
API	Festgelegte Schnittstelle, über die Programme Daten austauschen.	Frontend und Backend können unabhängig entwickelt und dennoch verbunden werden.
Branch	Eigener Entwicklungszeitweig eines Repositories.	Änderungen entstehen getrennt vom stabilen Hauptstand und lassen sich sicher prüfen.
Checkout / Switch	Wechsel zu einem anderen Branch oder Stand.	Du arbeitest anschließend wirklich im vorgesehenen Entwicklungszeitweig.
Clone	Lokale Kopie eines entfernten Repositories.	Damit kannst du Dateien auf deinem Rechner bearbeiten und testen.
Commit	Gespeicherter, beschriebener Zwischenstand ausgewählter Änderungen.	Kleine Commits machen Entwicklung nachvollziehbar und Fehler leichter rückgängig.
Commit Message	Kurze Nachricht, die Zweck und Art eines Commits erklärt.	Andere erkennen schnell, was geändert wurde und warum.
Conflict / Mergekonflikt	Git kann Änderungen an derselben Stelle nicht automatisch zusammenführen.	Der Konflikt verlangt eine bewusste Entscheidung über die gültige Fassung.
Definition of Done	Gemeinsame Qualitätsliste, die für jede fertige Aufgabe gilt.	„Fertig“ bedeutet dann mehr als „funktioniert bei mir“.
Fork	Eigene serverseitige Kopie eines fremden Repositories.	Du kannst unabhängig arbeiten und später

Begriff	Bedeutung	Wozu gibt es das?
		Änderungen vorschlagen.
Issue	Nachverfolgbare Aufgabe, Frage oder Fehlermeldung im Repository.	Es klärt Problem, Ziel und Prüfkriterien, bevor Code entsteht.
Label	Schlagwort an einem Issue oder Pull Request, etwa bug, frontend oder security.	Aufgaben lassen sich filtern und schneller einordnen.
Merge	Zusammenführen freigegebener Änderungen aus einem Branch.	Geprüfte Arbeit gelangt kontrolliert in den gemeinsamen Hauptstand.
Milestone	Gruppe von Issues und Pull Requests für ein gemeinsames Zwischenziel.	Der Fortschritt eines Sprints oder Releases wird sichtbar.
Pull	Holt entfernte Änderungen und integriert sie in den aktuellen lokalen Branch.	Dein Arbeitsstand bleibt mit der gemeinsamen Version synchron.
Pull Request (PR)	Vorschlag, einen Branch nach Prüfung zusammenzuführen.	Code, Tests und Begründungen können gemeinsam besprochen werden.
Push	Überträgt lokale Commits zum entfernten Repository.	Erst dadurch werden deine Commits für Team und Lehrkraft sichtbar.
README	Startdokument eines Repositories mit Zweck, Einrichtung und Nutzung.	Neue Teammitglieder können das Projekt verstehen und starten.
Repository (Repo)	Projektordner mit Dateien und vollständiger Git-Versionsgeschichte.	Code, Dokumentation, Aufgaben und Zusammenarbeit bleiben an einem Ort.
Review	Systematische Prüfung einer Änderung durch andere.	Feedback verbessert Qualität und macht Entscheidungen nachvollziehbar.
Stage / Staging Area	Auswahl der Änderungen, die in den nächsten Commit gehören.	Du kannst zusammengehörige Änderungen gezielt bündeln.
Tag / Release	Markierung beziehungsweise veröffentlichte Fassung eines bestimmten Stands.	Eine demonstrierte Version bleibt eindeutig auffindbar.

Begriff	Bedeutung	Wozu gibt es das?
User Story	Kurze Anforderung aus Sicht einer Nutzerrolle: Wer möchte was und wozu?	Technik wird vom tatsächlichen Bedarf aus geplant.
Workflow / GitHub Actions	Automatischer Ablauf, zum Beispiel Tests nach jedem Push.	Fehler und Qualitätsprobleme werden früh und reproduzierbar erkannt.
Agentenloop	Wiederholung aus Ziel verstehen, planen, Werkzeug nutzen, Ergebnis prüfen und weiterentscheiden.	Er unterscheidet einen handelnden Agenten von einer einzelnen Textantwort.
Embedding	Zahlenvektor, der inhaltliche Ähnlichkeit maschinenlesbar repräsentiert.	Ähnliche Fragen und Wissensabschnitte können gefunden werden.
LLM	Großes Sprachmodell, das Sprache verarbeitet und erzeugt.	Es formuliert Erklärungen, ist aber ohne Quellen und Prüfungen nicht automatisch zuverlässig.
Mock	Kontrollierte Attrappe für einen noch fehlenden oder teuren Systemteil.	Frontend und Tests können schon arbeiten, bevor das echte Modell angeschlossen ist.
Prompt	Eingabe mit Ziel, Kontext und Regeln für ein KI-Modell.	Eine klare Aufgabenstellung verbessert die Zusammenarbeit, ersetzt aber keine Prüfung.
Prompt Injection	Manipulative Anweisung in Eingaben oder Quellen, die Systemregeln umgehen soll.	Der Agent muss fremde Inhalte als Daten behandeln und Rechte begrenzen.
RAG	Vor einer Antwort werden passende Wissensabschnitte gesucht und dem LLM als Kontext gegeben.	Antworten können kursbezogener und mit Quellen belegbar werden.
Sandbox	Technisch isolierte Umgebung mit strengen Ressourcen- und Zugriffsgrenzen.	Unbekannter Schülercode darf das Hauptsystem nicht gefährden.
Schema	Formale Beschreibung erlaubter Datenfelder und Datentypen.	Fehlerhafte Eingaben und Ausgaben werden früh erkannt.

Begriff	Bedeutung	Wozu gibt es das?
Wissensgraph	Netz aus Knoten und benannten Beziehungen.	Voraussetzungen und Lernwege werden erklärbar und abfragbar.

Mini-Übung: Wähle drei Begriffe aus einem aktuellen Issue oder Pull Request. Erkläre jeden ohne Fachwort, zeige ihn direkt im Repository und beschreibe, welches Problem er im Team löst.

Quellen

OpenAI Agents: <https://developers.openai.com/api/docs/guides/agents>

MDN Accessibility:

https://developer.mozilla.org/docs/Learn_web_development/Core/Accessibility/HTML

React: <https://react.dev/learn/build-a-react-app-from-scratch>

OWASP Prompt Injection:

https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html